

Лекция 8: Введение в глубокое обучение и Сверточные нейронные сети (CNN)

Эта лекция посвящена основам глубокого обучения и сверточным нейронным сетям (CNN), которые произвели революцию в области компьютерного зрения. Мы рассмотрим ключевые концепции, архитектуры и методы оптимизации, лежащие в основе этих мощных моделей машинного обучения.

и by Иван Копылов

Определение глубокого обучения

1

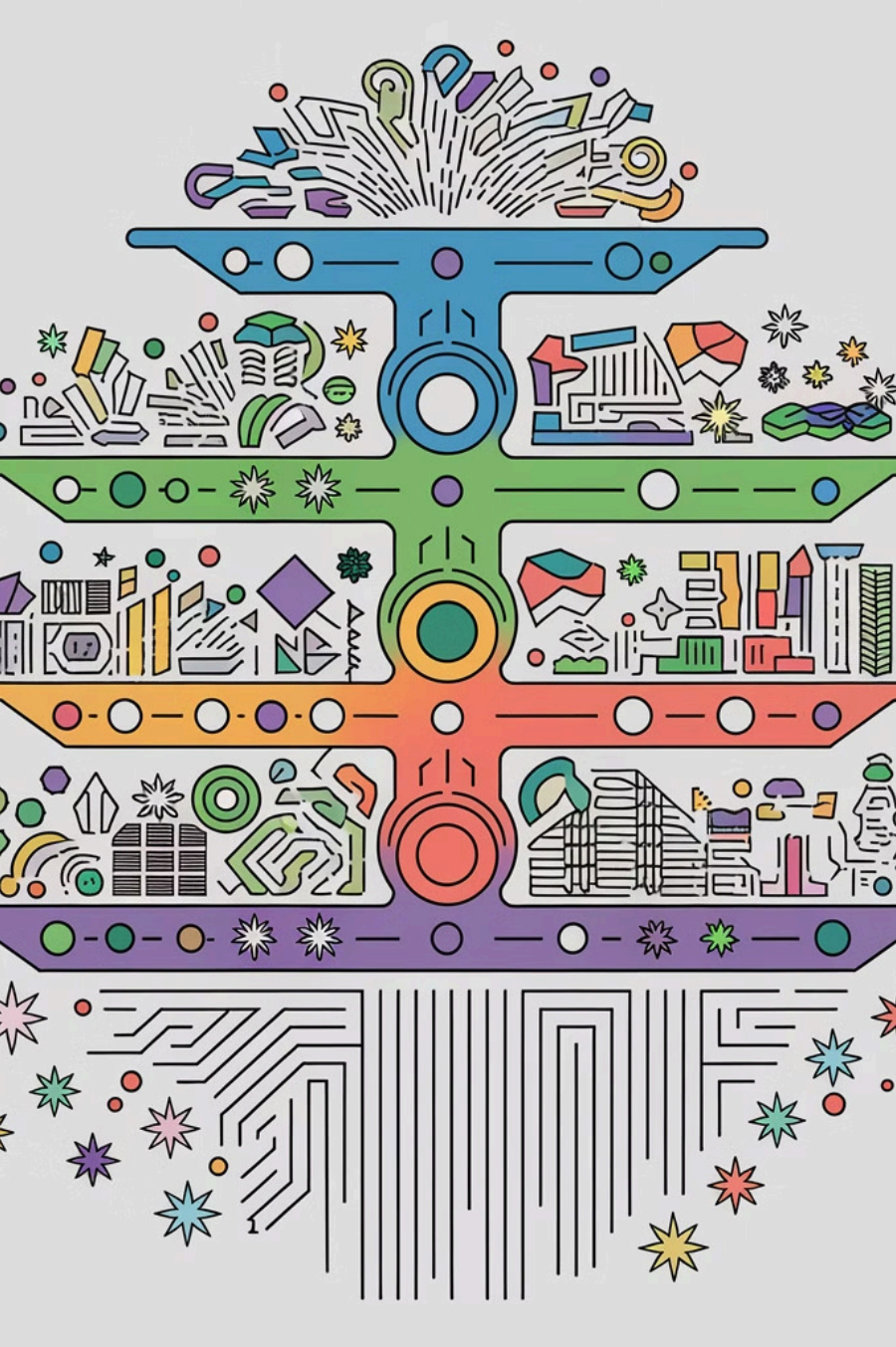
Глубокое обучение - подраздел машинного обучения, использующий многослойные искусственные нейронные сети для автоматического извлечения признаков из данных.

2

В отличие от классических методов, глубокие нейронные сети способны самостоятельно обучаться иерархическим структурам данных.

3

Формально, глубокое обучение - оптимизация параметров сложных нелинейных моделей, состоящих из большого количества слоев, путем минимизации функции потерь на обучающем наборе данных.



Отличия от классических методов машинного обучения

Классические методы

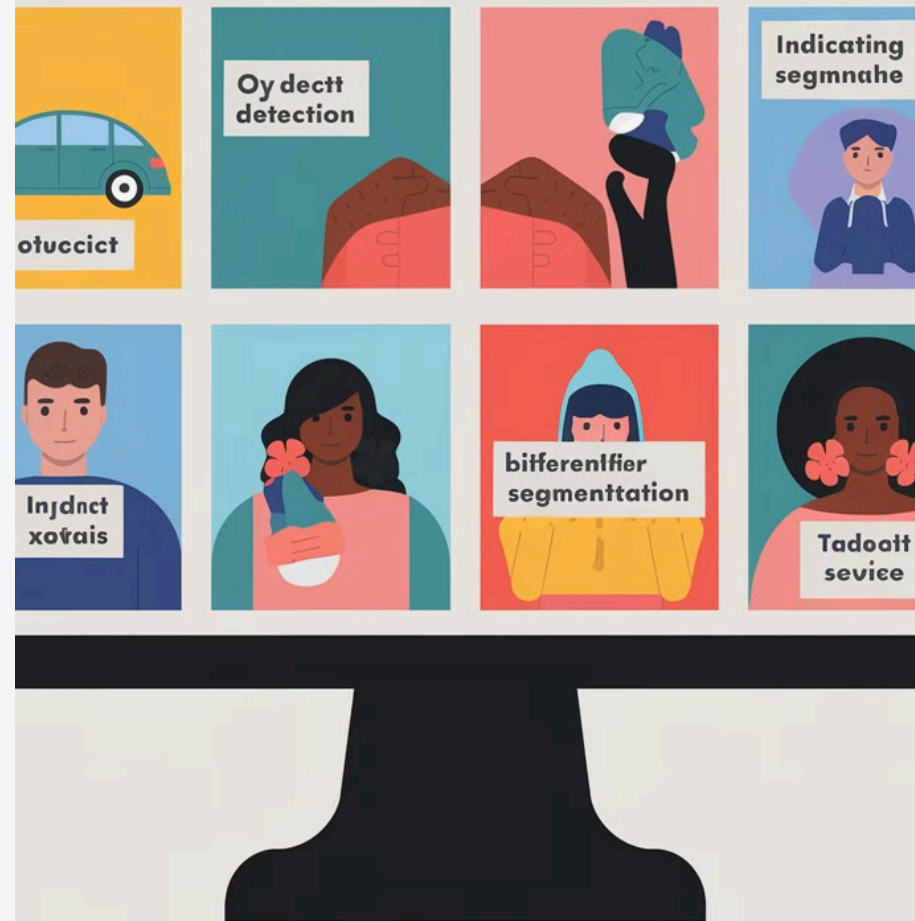
Ручное извлечение признаков (HOG, LBP, SIFT/SURF).
Ограничения в улавливании сложных нелинейных зависимостей. Требуют значительных экспертных знаний.

Глубокое обучение

Самостоятельное обучение представлениям данных.
Иерархическая структура от низкоуровневых к абстрактным признакам. Более глубокое понимание данных и высокие показатели качества.

Роль глубокого обучения в компьютерном зрении

- 1 Автоматическое извлечение признаков: CNN обучаются выявлять релевантные признаки напрямую из данных.
- 2 Иерархическая репрезентация: Многослойная архитектура позволяет строить сложные представления от простых краев до объектных шаблонов.
- 3 Обобщающая способность: Глубокие модели демонстрируют высокую способность к обобщению на новых данных.
- 4 Масштабируемость: Благодаря параллельным вычислениям на GPU, глубокие нейронные сети эффективно обучаются на огромных наборах данных.



Математическая модель нейрона (перцептрона)

Искусственный нейрон (перцептрон) – базовый элемент нейронной сети, имитирующий работу биологического нейрона. Он вычисляет взвешенную сумму входных сигналов с последующим применением нелинейной функции активации.

Формально, нейрон определяется уравнением:

$$y = \varphi(\sum_{i=1}^n w_i x_i + b)$$

где x_i – входные сигналы, w_i – веса, b – смещение, φ – функция активации, y – выход нейрона.

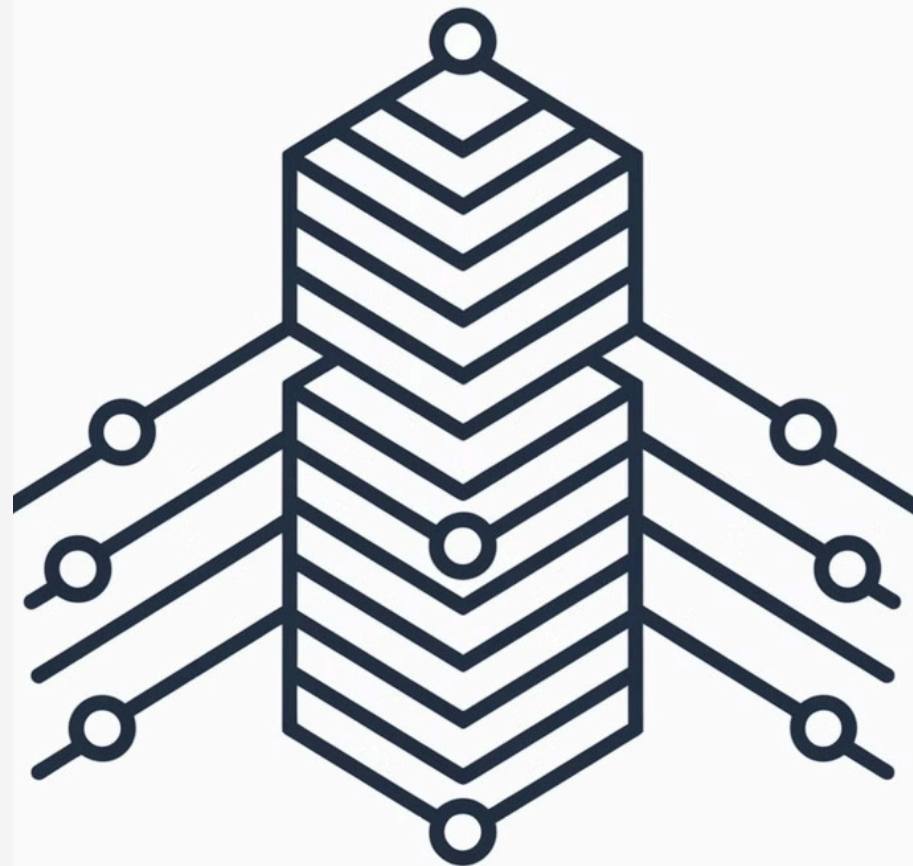
Многослойные перцептроны (MLP)

Многослойный перцептрон (MLP) - архитектура, состоящая из входного слоя, одного или нескольких скрытых слоев и выходного слоя. Каждый скрытый слой состоит из множества нейронов, принимающих на вход выходы предыдущего слоя.

Общая схема MLP записывается следующим образом:

$$h^{(l)} = \varphi^{(l)}(W^{(l)} h^{(l-1)} + b^{(l)})$$

где $h^{(l-1)}$ - вектор входов для l -го слоя, $W^{(l)}$ - матрица весов, $b^{(l)}$ - вектор смещений, $\varphi^{(l)}$ - функция активации, $h^{(l)}$ - вектор выходов l -го слоя.



Функции активации

Сигмоида (Sigmoid)

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Ограничивает выход значениями (0, 1).

Удобна для моделирования вероятностей.

Гиперболический тангенс (Tanh)

$$\tanh(z) = (e^z - e^{(-z)}) / (e^z + e^{(-z)})$$

Преобразует вход в диапазон от -1 до 1. Может ускорить сходимость.

ReLU

$$\text{ReLU}(z) = \max(0, z)$$

Стандартная функция для скрытых слоев. Помогает смягчить проблему исчезающих градиентов.

Алгоритм обратного распространения ошибки

Обучение нейронной сети основывается на методе обратного распространения ошибки (backpropagation). Суть алгоритма в вычислении градиентов функции потерь по отношению к параметрам сети и последующем обновлении этих параметров методом градиентного спуска.

Обновление параметров происходит по правилу:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \partial L / \partial \theta$$

где θ - вектор параметров, η - скорость обучения, L - функция потерь.

Оптимизация: градиентный спуск, SGD, Adam

- 1 Стохастический градиентный спуск (SGD): Вычисляет градиенты на основе мини-батчей данных, ускоряя итерации и уменьшая потребление памяти.
- 2 Adam: Сочетает идеи адаптивного изменения скорости обучения и импульса, обеспечивая быструю сходимость.

Пример реализации простого MLP с использованием TensorFlow (Keras):

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation
from tensorflow.keras.optimizers import Adam

model = Sequential([
    Dense(128, input_shape=(784,)),
    Activation('relu'),
    Dense(64),
    Activation('relu'),
    Dense(10),
    Activation('softmax')
])

model.compile(optimizer=Adam(learning_rate=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

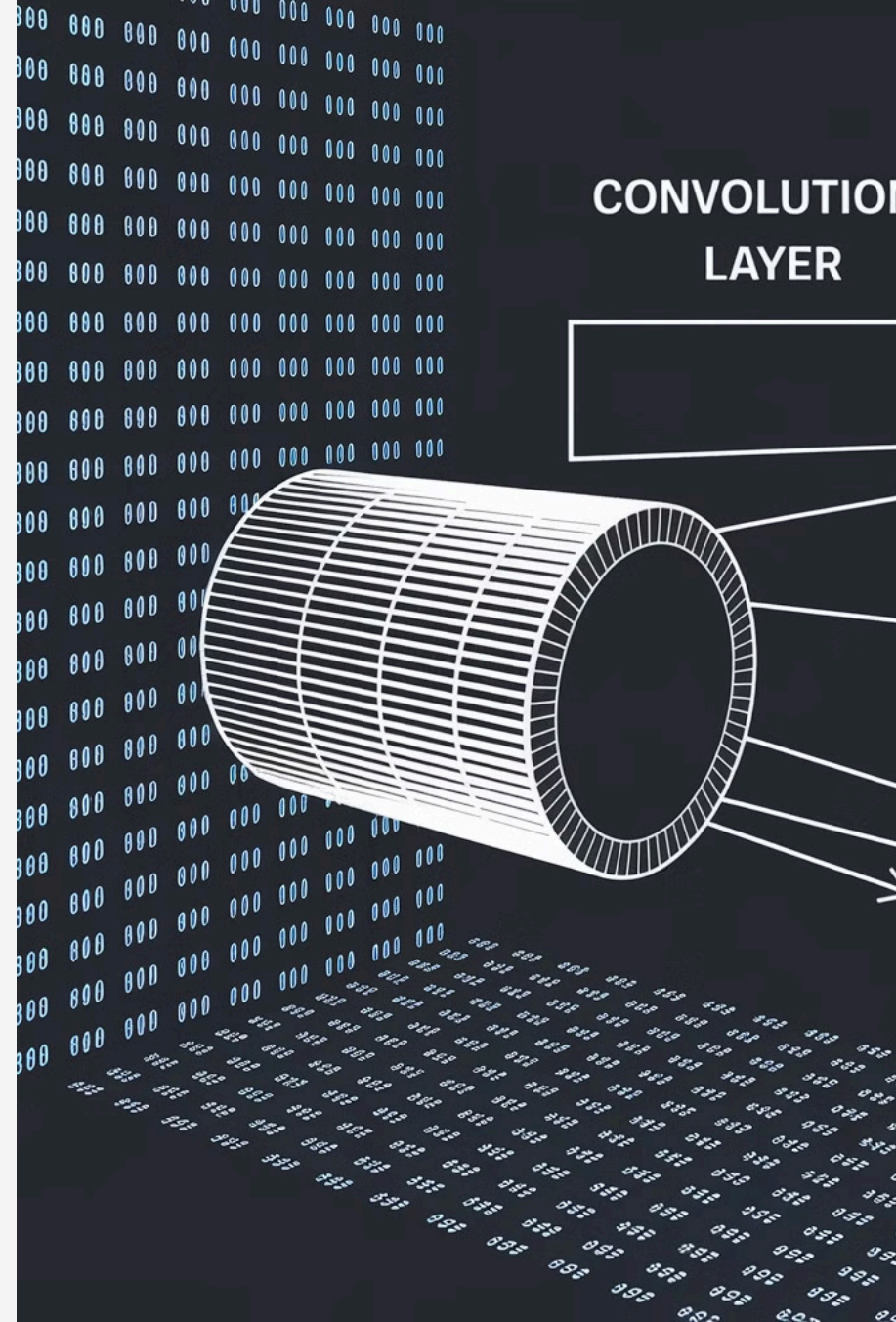
Сверточный слой

Сверточный слой - основной строительный блок CNN. В этом слое происходит свертка входного изображения с помощью набора фильтров. Каждый фильтр представляет собой матрицу небольшого размера, параметры которой обучаются в процессе оптимизации.

Операция свертки определяется как:

$$S(i,j) = (I * K)(i,j) = \sum(m) \sum(n) I(i+m, j+n) K(m, n)$$

где S - выход сверточного слоя, I - входное изображение, K - фильтр.



Функции активации в сверточных слоях

После применения операции свертки к результату обычно применяется функция активации, например, ReLU (Rectified Linear Unit):

$$\text{ReLU}(z) = \max(0, z)$$

Данная функция обеспечивает нелинейность, что позволяет модели обучать более сложные зависимости. Использование нелинейных функций активации является обязательным условием для успешного обучения глубоких архитектур.

Субдискретизация и пулинг (Pooling)

Max Pooling

Выбирается максимальное значение в пределах заданного окна.

Average Pooling

Вычисляется среднее значение по окну.

Для max pooling с окном $p \times p$ можно записать:

$$P(i,j) = \max \{ S(m,n) \mid m,n \in \text{окно вокруг } (i,j) \}$$

Пулинг сокращает количество параметров и вычислительных затрат, а также способствует инвариантности к небольшим пространственным сдвигам.

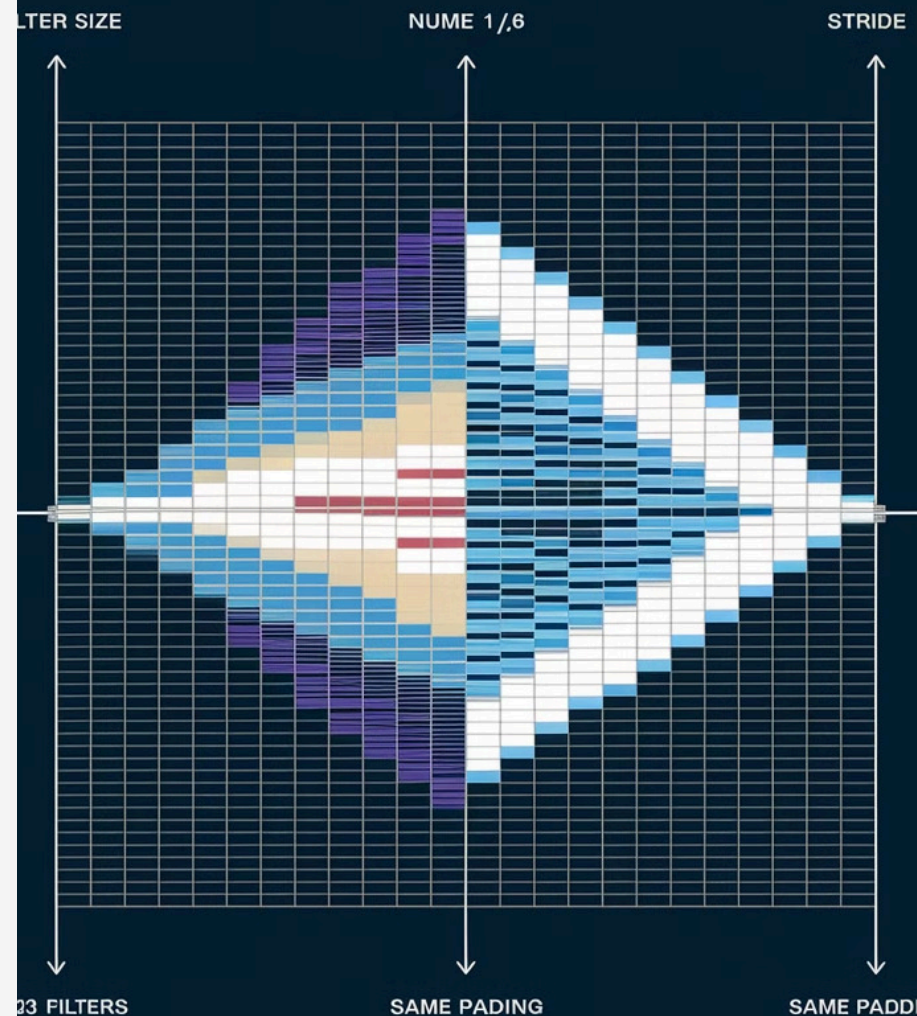
Операция свертки

Операция свертки является дискретным аналогом интегрального оператора свертки, широко используемого в обработке сигналов. При свертке фильтр проходит по всему изображению с заданным шагом (stride).

Если шаг равен 1, фильтр сдвигается на один пиксель за итерацию, а при шаге 2 – сдвигается на два пикселя, что приводит к уменьшению размерности выходного представления. Для контроля размерности также применяется понятие «паддинга» (padding), при котором границы изображения дополняются нулями.

Параметры свертки: фильтры, шаг, паддинг

- 1 Размер ядра (kernel size): Определяет размер фильтра (например, 3×3 или 5×5).
- 2 Количество фильтров: Определяет, сколько различных признаков карт будет сформировано.
- 3 Stride (шаг): Определяет величину сдвига фильтра по входному изображению.
- 4 Padding (отступ): Добавление дополнительного заполнения вокруг изображения для сохранения пространственных размеров после свертки.

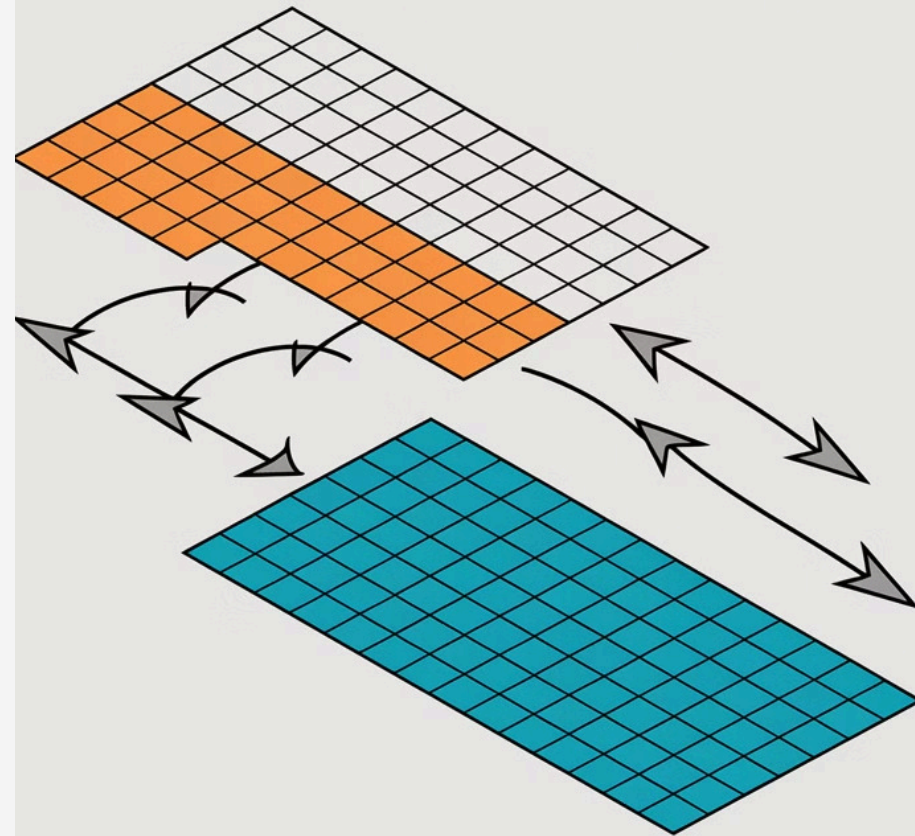


Математическая запись сверточной операции

Для входного тензора I размером $H \times W \times C$ и набора F фильтров K размера $k_h \times k_w \times C$ операция свертки записывается как:

$$S_f(i,j) = \sum_{c=1 \text{ to } C} \sum_{u=1 \text{ to } k_h} \sum_{v=1 \text{ to } k_w} I_c(i+u-1, j+v-1) K_{f,c}(u,v) + b_f$$

где $S_f(i,j)$ – значение на позиции (i,j) для f -го фильтра, b_f – смещение для f -го фильтра, а c – индекс канала.



Иерархическая организация сверточных сетей

Начальные сверточные слои

Выявляют низкоуровневые признаки, такие как края, углы и текстуры.

Средние сверточные слои

Объединяют низкоуровневые признаки в более сложные паттерны, например, формы и части объектов.

Глубокие сверточные слои

Формируют высокоуровневые представления, соответствующие целостным объектам или семантическим категориям.

Полносвязанные слои

После извлечения признаков выполняется классификация, где выходные признаки преобразуются в итоговое вероятностное распределение по классам.



Пример реализации CNN для классификации изображений

Рассмотрим реализацию простой сверточной нейронной сети с использованием TensorFlow (Keras):

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential([
    Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(32, 32, 3)),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(64, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])

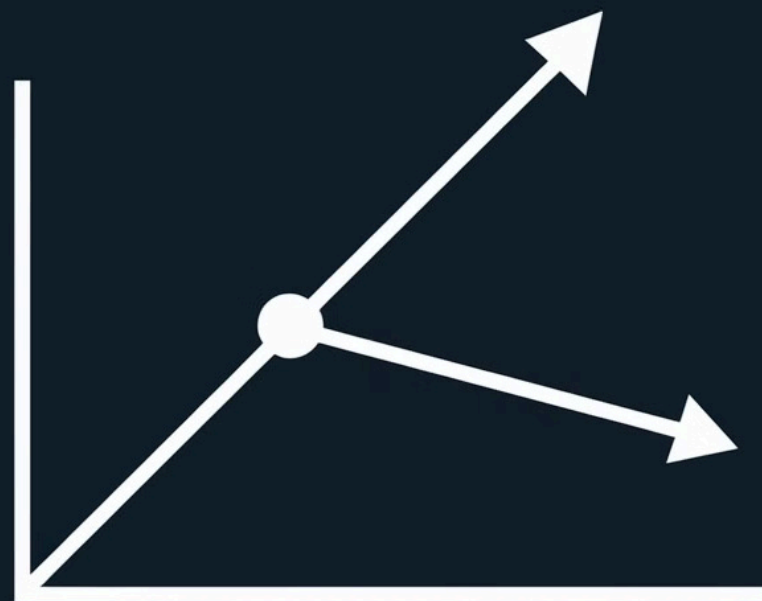
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Обучение сверточных сетей: функции потерь и оптимизация

При обучении CNN основное внимание уделяется выбору функции потерь и метода оптимизации. Для задач классификации часто используется функция кросс-энтропии:

$$L = -\sum_{i=1 \text{ to } N} \sum_{c=1 \text{ to } C} t_{(i,c)} \log(y_{(i,c)})$$

где $t_{(i,c)}$ – истинная метка (one-hot представление) для i -го примера, $y_{(i,c)}$ – предсказанная вероятность принадлежности к c -му классу, а N – количество примеров. Для оптимизации параметров применяются алгоритмы, такие как SGD, Adam или RMSProp.



Регуляризация и методы предотвращения переобучения в CNN

1

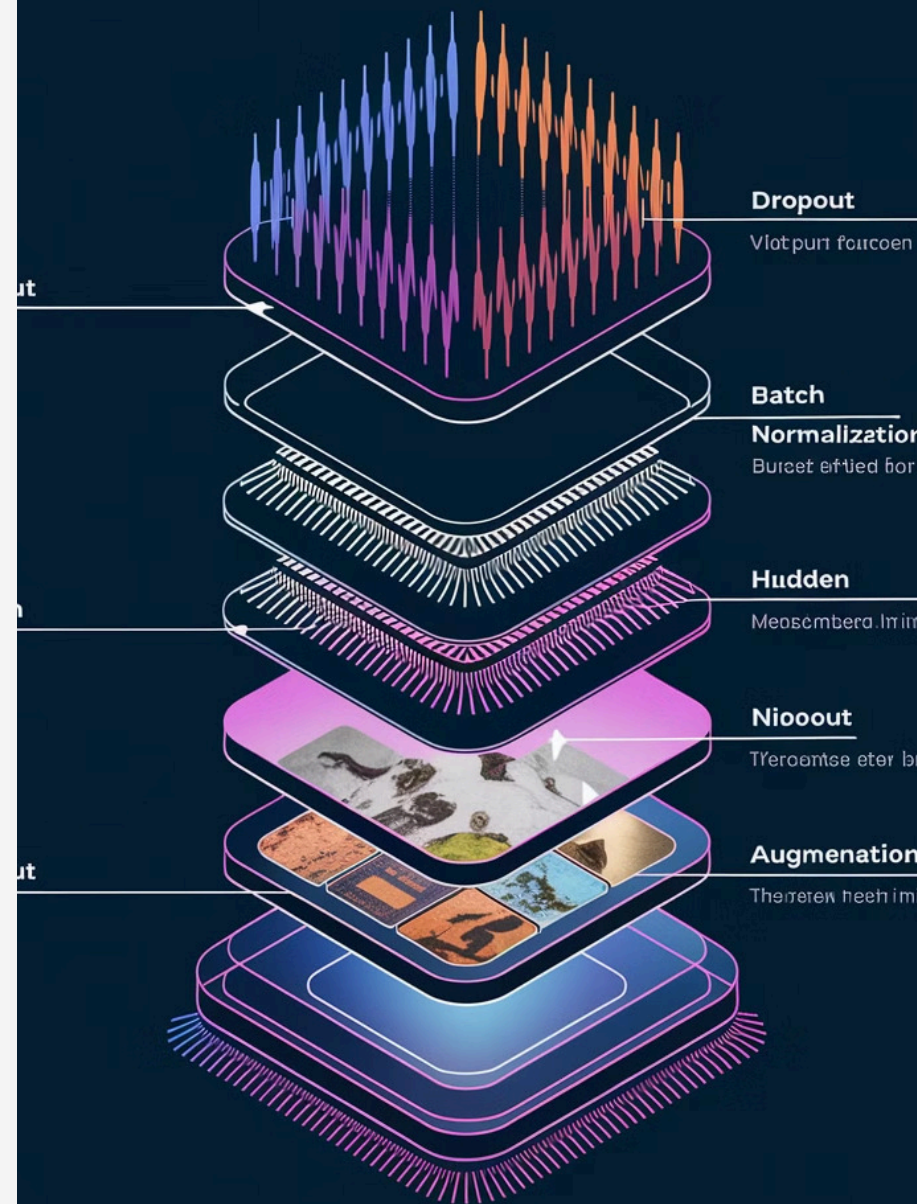
Dropout: На этапе обучения случайно "выключаются" нейроны с определенной вероятностью, что снижает зависимость модели от конкретных признаков.

2

Batch Normalization: Нормализует распределение входов для каждого слоя, что стабилизирует и ускоряет обучение.

3

Аугментация данных: Применение трансформаций изображений увеличивает разнообразие обучающей выборки и способствует лучшей обобщающей способности модели.



Продвинутые варианты сверточных слоев

Dilated (разреженные) свертки

Позволяют расширить область рецепции без увеличения количества параметров за счёт введения промежутков между элементами фильтра.

Транспонированные свертки

Используются для увеличения размерности признаков карт, например, в задачах сегментации и генерации изображений.

Групповые и разделяемые свертки

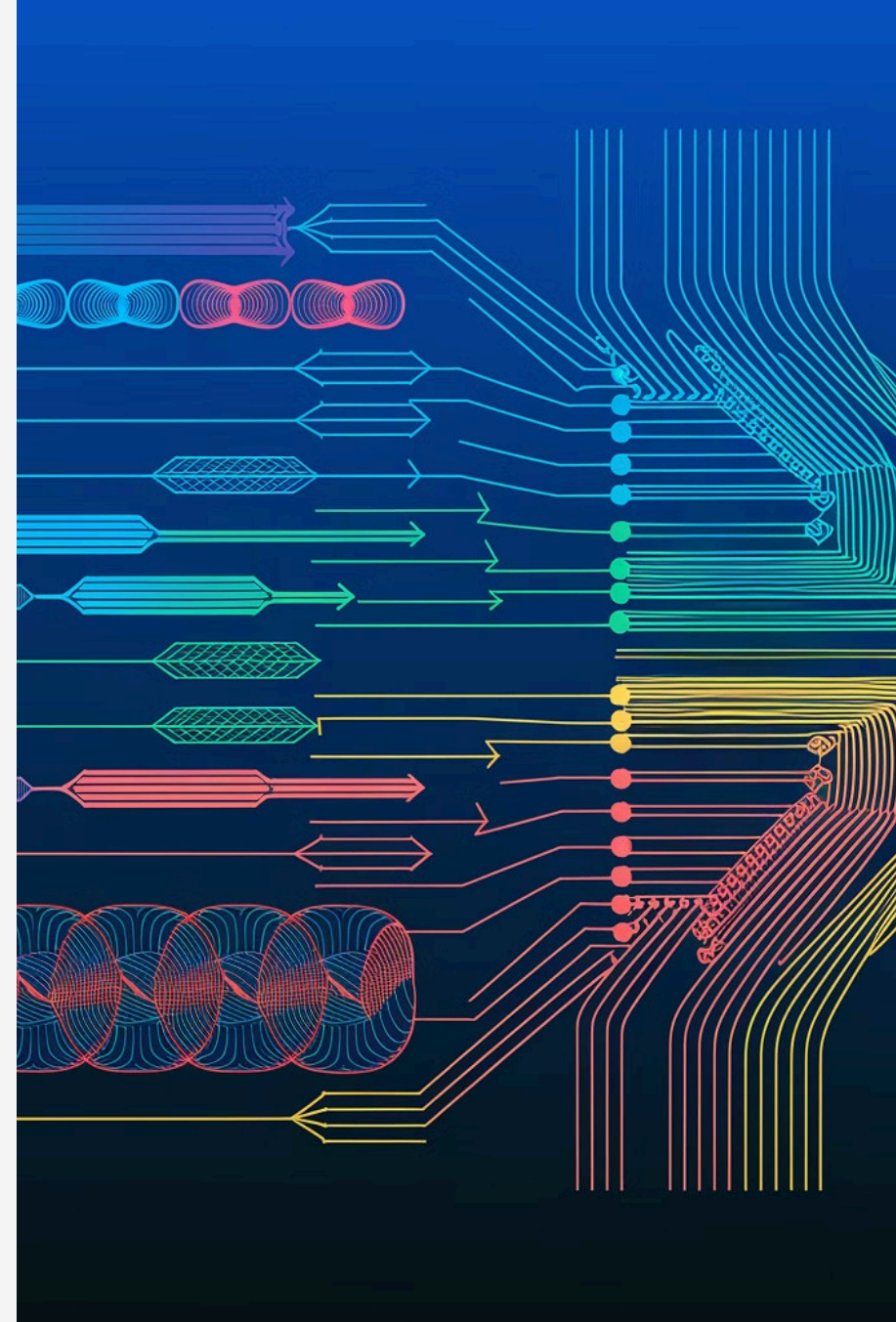
Направлены на уменьшение вычислительной сложности и количества параметров.

Математические аспекты обратного распространения в CNN

Обратное распространение в сверточных сетях включает вычисление градиентов для параметров фильтров. Если обозначить операцию свертки как $S = I * K$, то градиент функции потерь L по фильтру K определяется через правило цепочки:

$$\partial L / \partial K = I \star \partial L / \partial S$$

где $\star$ обозначает операцию корреляции, обратную свертке. При этом градиенты по входному изображению вычисляются аналогичным образом, что позволяет обновлять веса всех слоев сети.



Глубокие сверточные архитектуры

1

VGG-сети используют последовательные блоки сверточных слоев с небольшими ядрами (3×3) и периодическим применением max pooling.

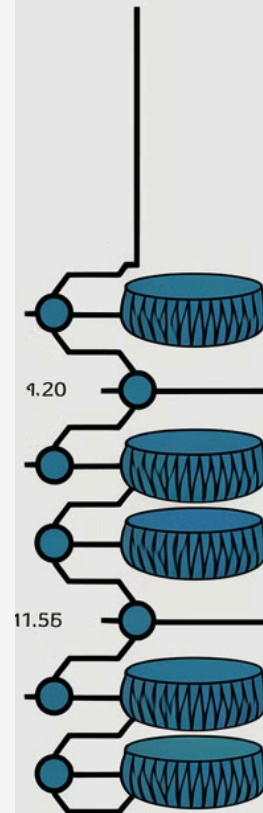
2

ResNet вводит концепцию остаточных (residual) связей, которые позволяют информации проходить через несколько слоев без существенного затухания градиента.

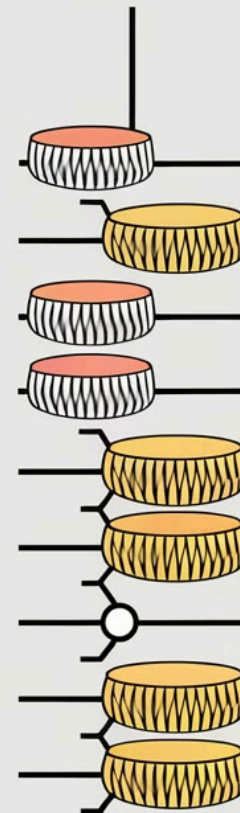
3

Inception предлагает использование параллельных сверточных слоев с различными размерами ядер в одном блоке.

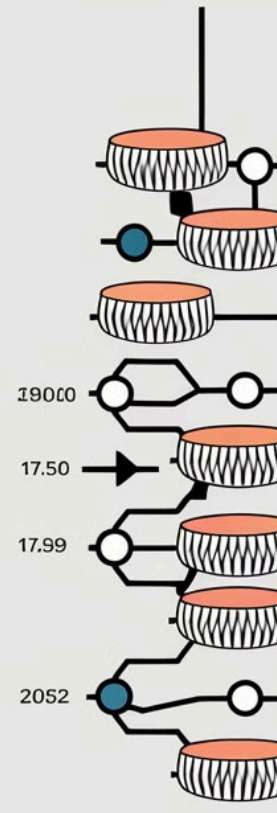
VGG



RESNET



INCEPTION



Примеры архитектур CNN

1

VGG-16/VGG-19:

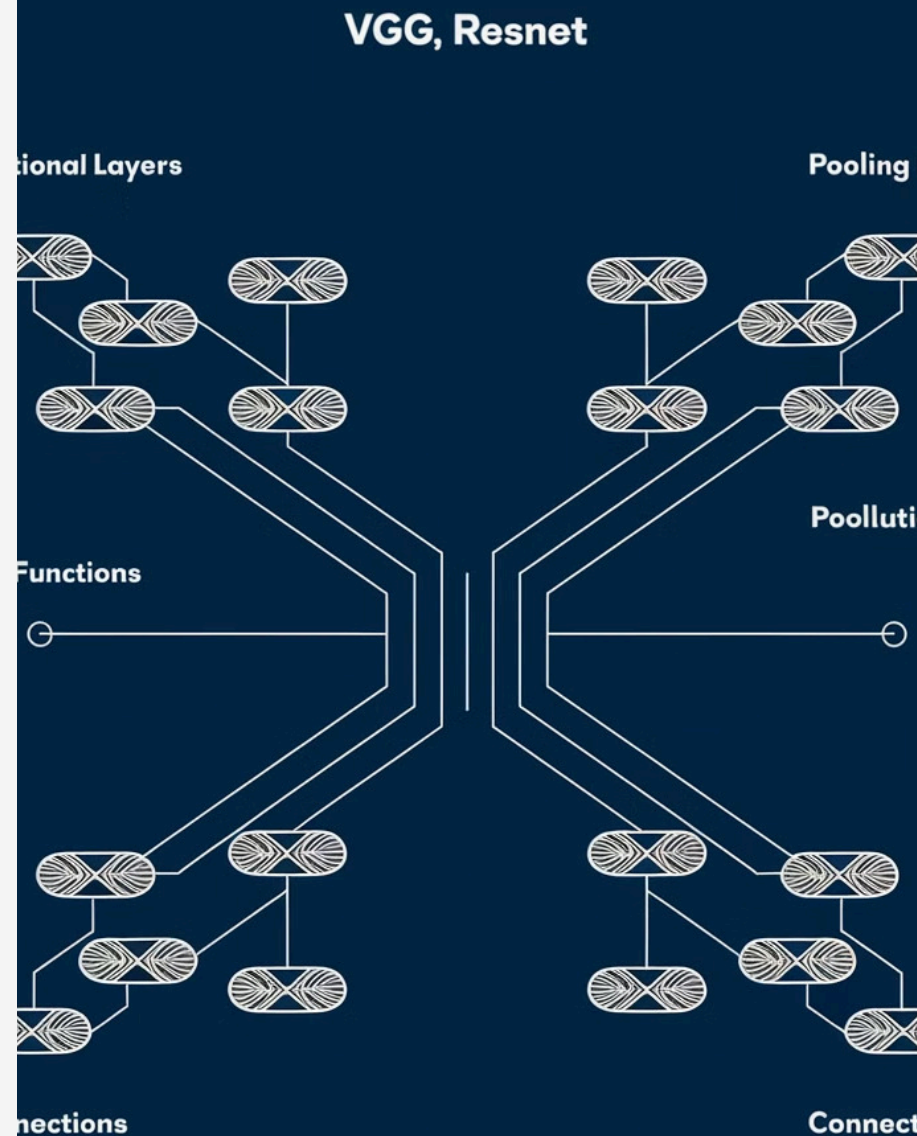
Последовательное применение сверточных блоков с небольшими ядрами, за которыми следуют полносвязанные слои.

2

ResNet-50/101/152: Введение остаточных связей позволяет строить очень глубокие сети без проблемы исчезающих градиентов.

3

Inception-v3: Архитектура использует параллельные пути обработки, объединяя результаты сверточных операций с разными размерами ядер.



Пример реализации архитектуры ResNet с использованием PyTorch

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1, downsample=None):
        super(ResidualBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.downsample = downsample

    def forward(self, x):
        identity = x
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.downsample is not None:
            identity = self.downsample(x)
        out += identity
        out = F.relu(out)
        return out

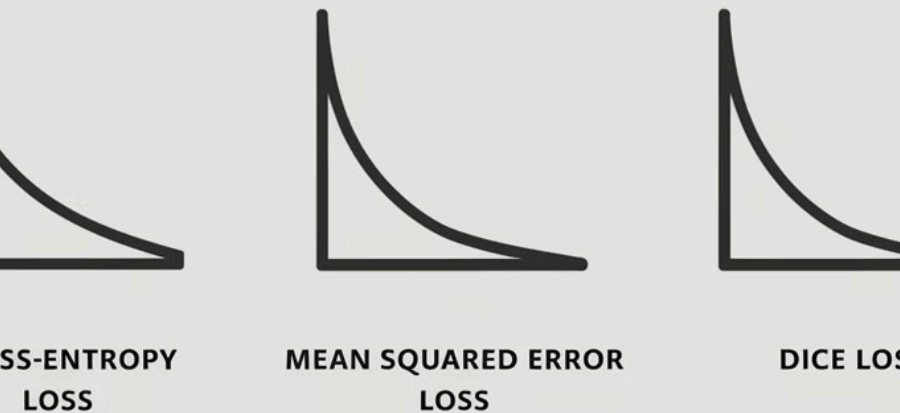
class ResNet(nn.Module):
    def __init__(self, block, layers, num_classes=10):
        super(ResNet, self).__init__()
        self.in_channels = 64
        self.conv = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
        self.bn = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        self.layer1 = self._make_layer(block, 64, layers[0])
        self.layer2 = self._make_layer(block, 128, layers[1], stride=2)
        self.layer3 = self._make_layer(block, 256, layers[2], stride=2)
        self.layer4 = self._make_layer(block, 512, layers[3], stride=2)
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(512, num_classes)

    def _make_layer(self, block, out_channels, blocks, stride=1):
        downsample = None
        if stride != 1 or self.in_channels != out_channels:
            downsample = nn.Sequential(
                nn.Conv2d(self.in_channels, out_channels, kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )
        layers = []
        layers.append(block(self.in_channels, out_channels, stride, downsample))
        self.in_channels = out_channels
        for _ in range(1, blocks):
            layers.append(block(out_channels, out_channels))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.relu(self.bn(self.conv(x)))
        x = self.maxpool(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.fc(x)
        return x

def resnet18(num_classes=10):
    return ResNet(ResidualBlock, [2, 2, 2, 2], num_classes)

# model = resnet18(num_classes=100)
```

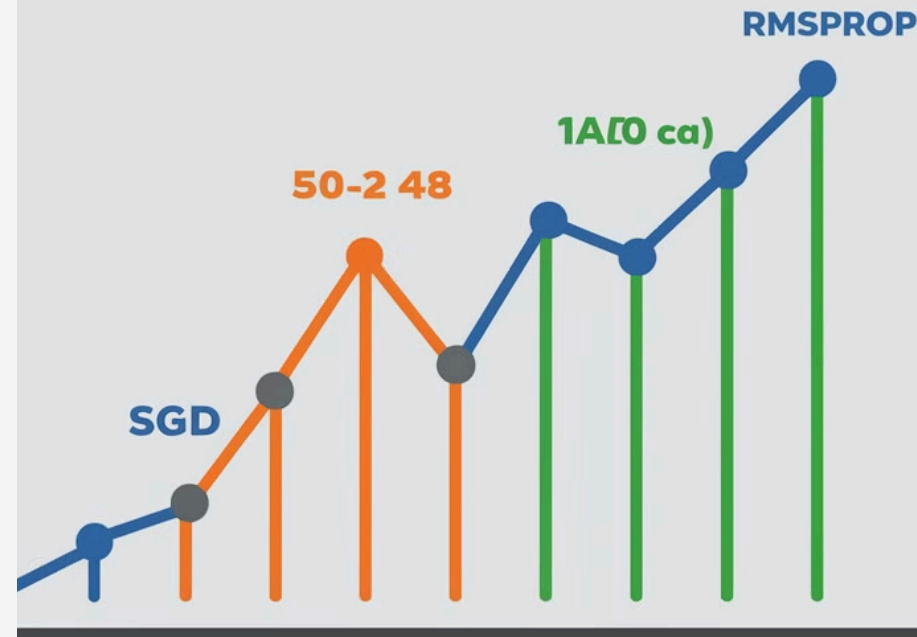


Выбор функции потерь

Для задач классификации часто используется кросс-энтропийная функция потерь, тогда как для задач регрессии или сегментации могут применяться другие функции, такие как MSE (mean squared error) или dice loss. Корректный выбор функции потерь существенно влияет на качество обученной модели.

Оптимизаторы и их настройки

В сверточных сетях оптимизаторы, такие как Adam, RMSProp и SGD с моментумом, широко используются для обеспечения устойчивого и быстрого обучения. Настройка гиперпараметров оптимизатора (скорость обучения, коэффициенты затухания) критична для достижения хорошей сходимости.



Регуляризация через Dropout и Batch Normalization

Dropout

Применяется в полносвязанных слоях для случайного отключения нейронов. Это позволяет бороться с переобучением и улучшает стабильность обучения.

Batch Normalization

Используется после сверточных слоев для нормализации активаций. Это ускоряет сходимость и стабилизирует процесс обучения.

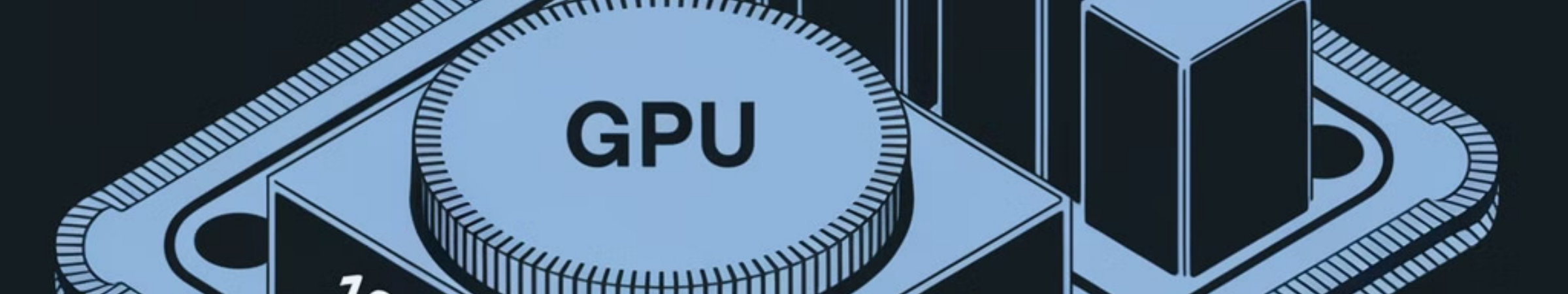
L2-регуляризация

Вводится через штраф на норму весов: $L_{total} = L_{data} + \lambda |W|_2^2$, где λ – коэффициент регуляризации.



Data Augmentation

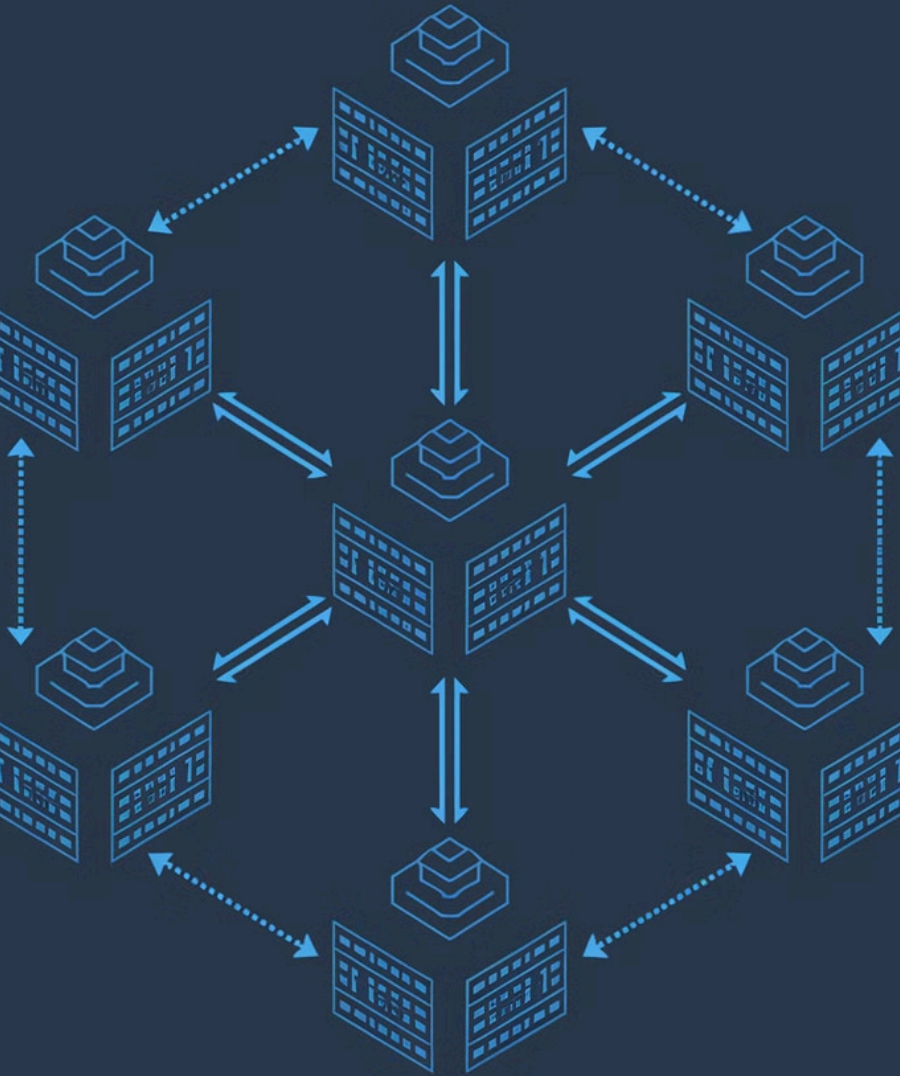
Аугментация данных играет ключевую роль в обучении CNN. Применяемые преобразования включают случайное вращение, масштабирование, отражение и изменение яркости, что помогает сети обобщать и уменьшать переобучение.



Аппаратное ускорение

Современные фреймворки для глубокого обучения оптимизированы для работы с GPU и TPU, что позволяет значительно ускорить обучение сверточных сетей за счет параллельных вычислений. Использование таких библиотек, как cuDNN, позволяет реализовывать эффективные алгоритмы свертки, что критически важно для обработки больших наборов изображений.

Параллелизм и распределенное обучение



1

Data Parallelism:

Разделение обучающей выборки на мини-батчи, обрабатываемые параллельно на нескольких GPU, с последующей синхронизацией градиентов.

2

Model Parallelism:

Разделение архитектуры сети на части, каждая из которых обучается на отдельном устройстве.

3

Синхронное и асинхронное обновление параметров: Выбор между синхронной и асинхронной схемами обновления влияет на скорость и стабильность сходимости модели.



Профилирование и оптимизация вычислительных графов

Современные фреймворки позволяют проводить детальное профилирование вычислительных графов, что помогает оптимизировать использование памяти и времени вычислений. Тонкая настройка параметров, таких как размер батча и порядок выполнения операций, способствует повышению эффективности обучения CNN.

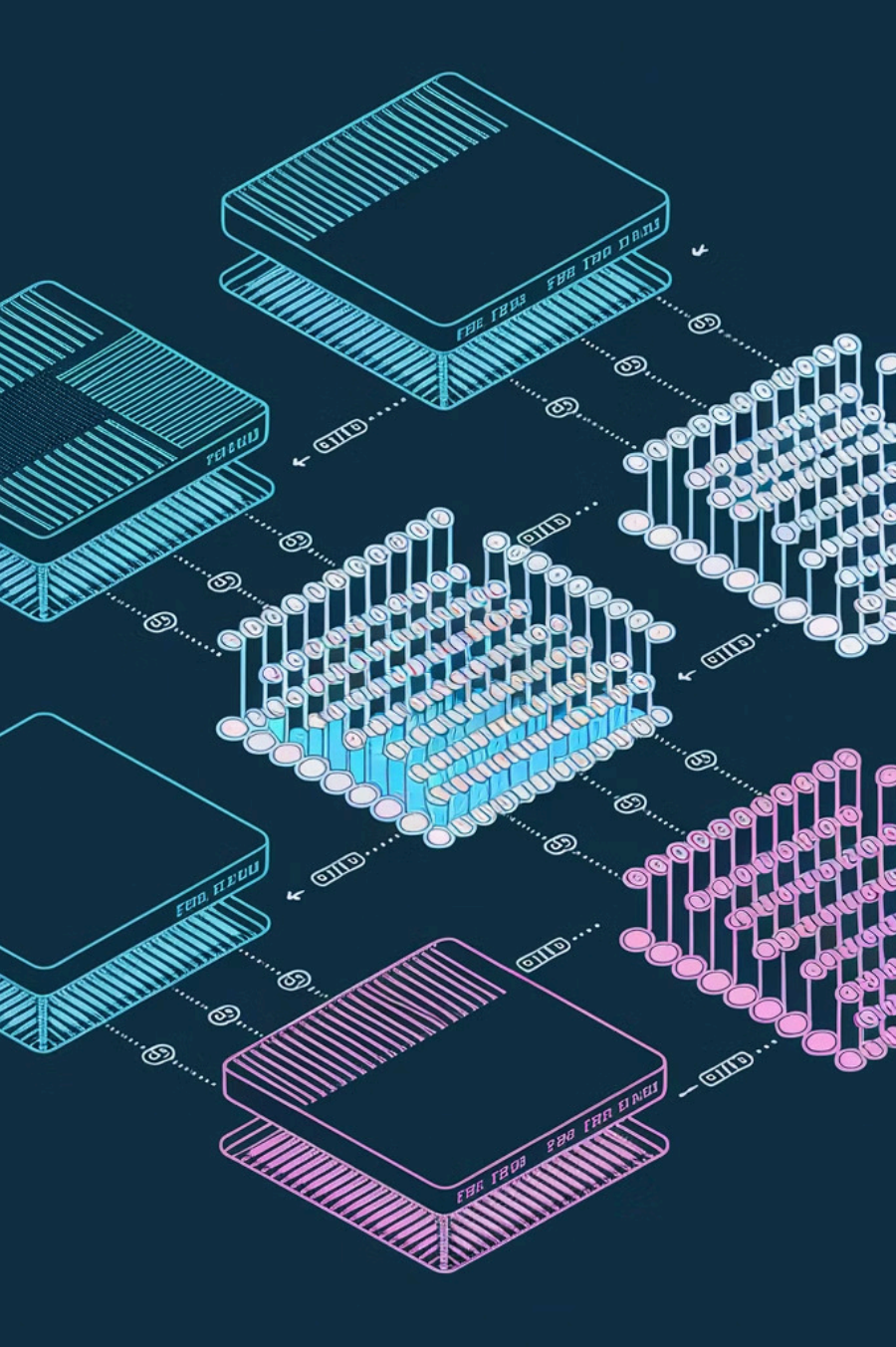
Адаптивные сверточные слои

Свертки с динамическими ядрами

Параметры фильтров вычисляются на лету на основе входных данных.

Attention-механизмы

Позволяют модели фокусироваться на наиболее важных регионах изображения.

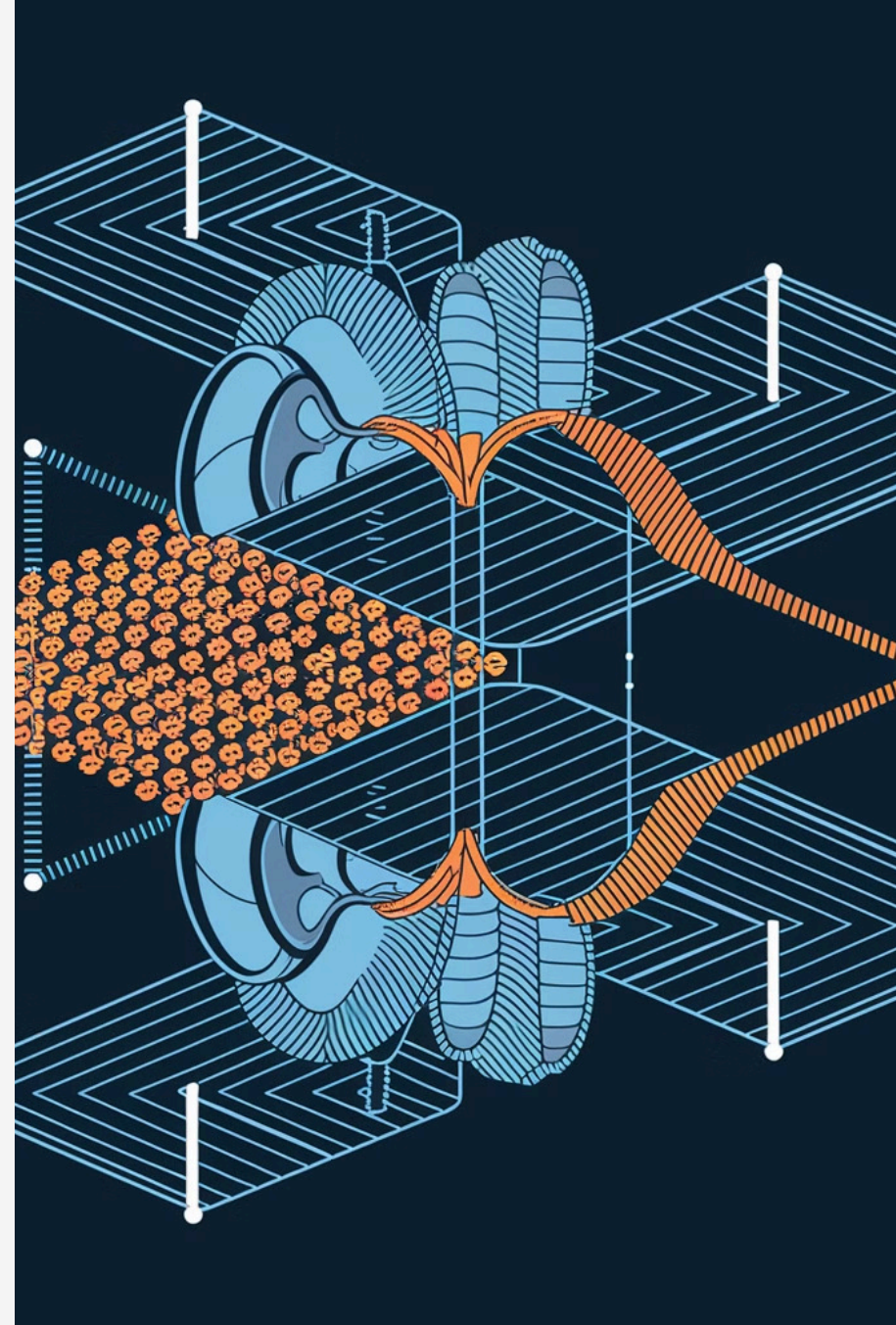


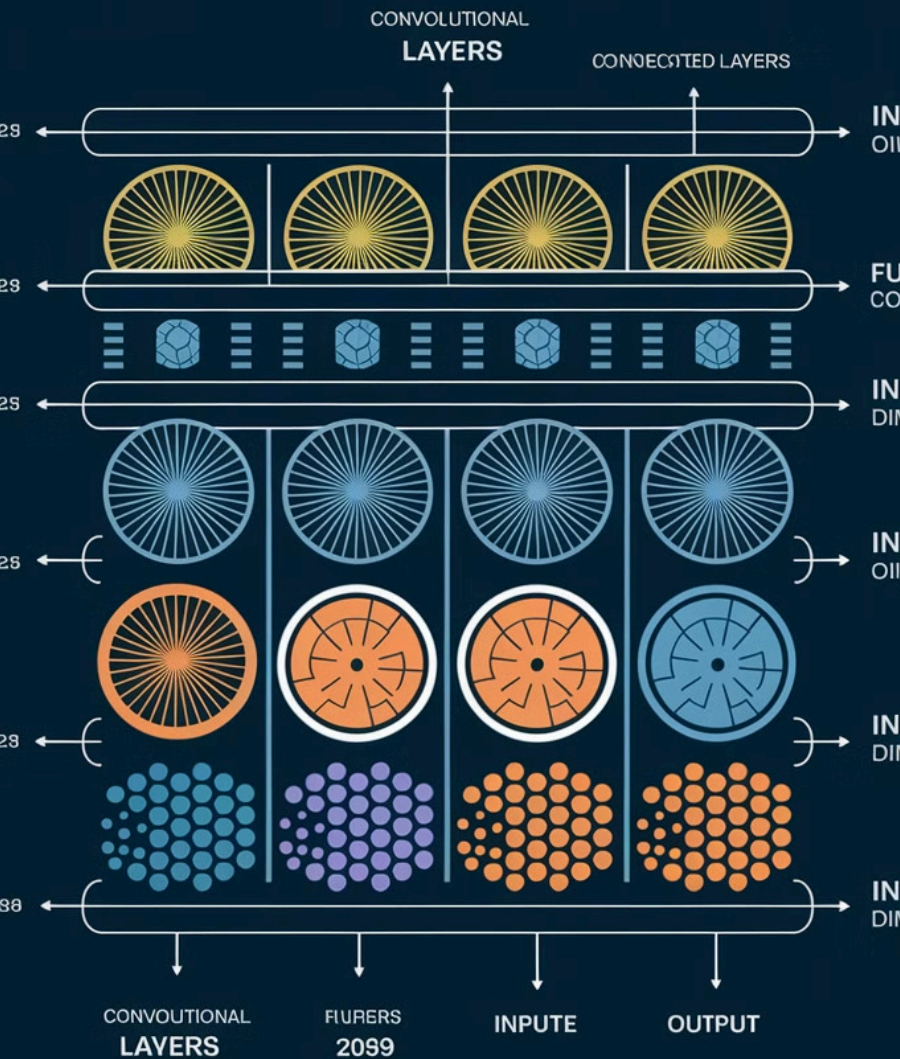
Модули Inception

Модули Inception позволяют объединять сверточные операции с разными размерами ядер в одном блоке. Это обеспечивает обработку информации на различных масштабах и позволяет модели захватывать как мелкие, так и крупные детали изображения.

Transposed Convolutions и Upsampling

Для задач, требующих восстановления пространственной размерности (например, сегментация или генерация изображений), используются транспонированные свертки. Эти слои позволяют увеличить размерность выходного тензора, что является обратной операцией к обычной свертке.

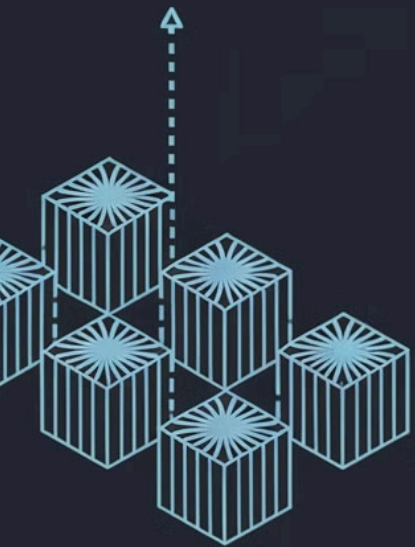




Комбинирование CNN с полносвязными слоями

После серии сверточных и пулинг-слоев извлечённые признаки передаются в один или несколько полносвязанных слоев, где происходит окончательная классификация. Преобразование пространственных признаков в одномерный вектор (flattening) позволяет использовать стандартные методы обучения многослойного перцептрона для принятия окончательного решения.

CNN Architecture



tenine consists re effects
multiensality reduction

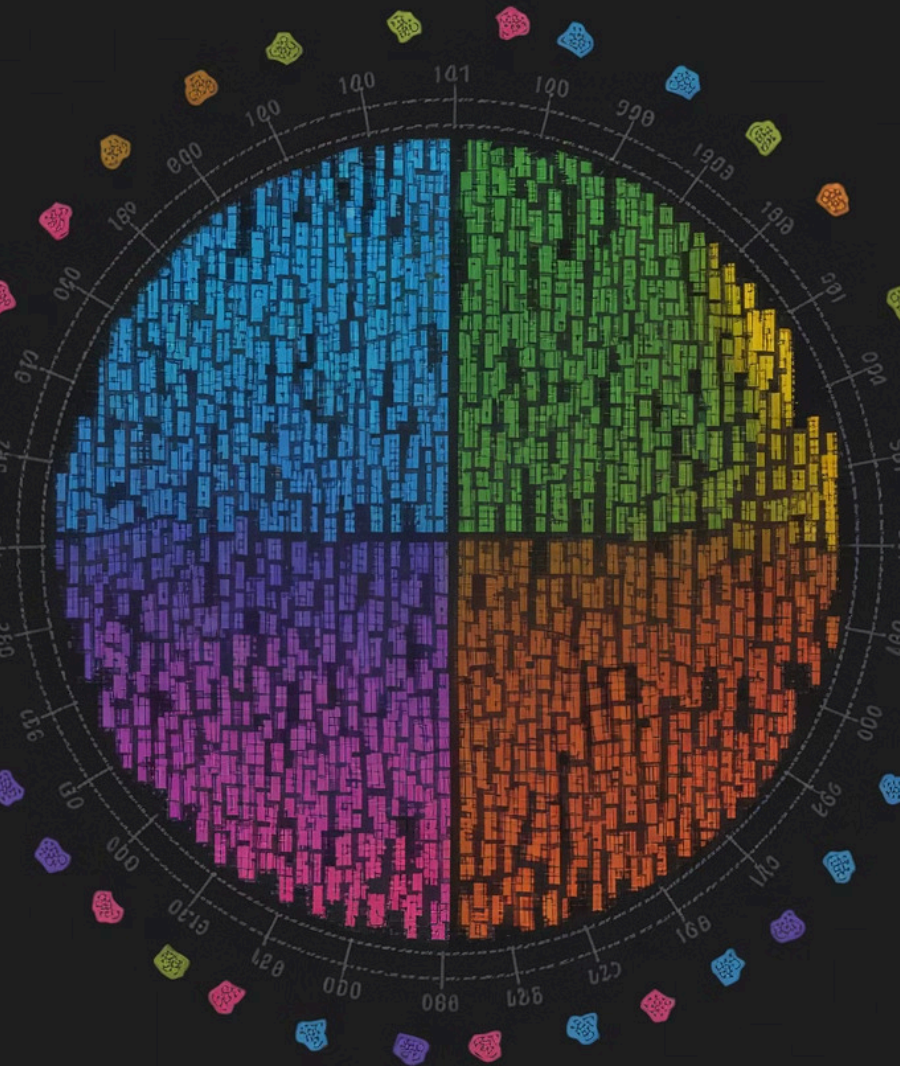
Global Average Pooling



Global Average of meth
dimensional layers

Использование глобального пулинга

Глобальный average pooling или max pooling часто применяется вместо традиционного flattening, что позволяет существенно снизить количество параметров в модели и повысить её устойчивость к переобучению. Глобальный average pooling вычисляет среднее значение по каждому каналу, превращая тензор признаков в вектор фиксированной длины.



Слияние признаков карт

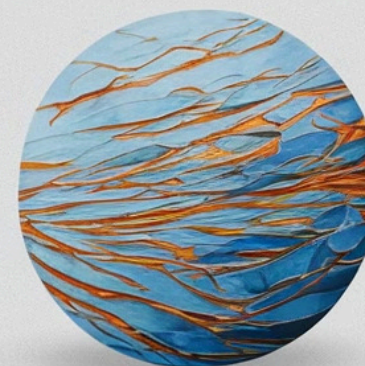
В некоторых архитектурах применяется операция concatenation или суммирования признаков карт из различных слоев, что позволяет объединить информацию, полученную на разных уровнях абстракции. Такие подходы способствуют лучшему представлению данных и улучшают качество итоговой классификации.

Выбор количества фильтров и размерности

Количество фильтров в сверточном слое определяет, сколько различных признаков карт будет получено. Увеличение числа фильтров позволяет захватывать больше различных паттернов, однако приводит к увеличению вычислительной сложности. Размер фильтра обычно подбирается эмпирически; часто используются фильтры размером 3×3 или 5×5 , поскольку они обеспечивают баланс между локальностью и вычислительной эффективностью.



3x3



5x5



7x7

Stride и padding

Stride (шаг)

Влияет на размер выходных признаковых карт. При увеличении шага уменьшается размер выходного тензора.

Padding (отступ)

"Same padding" позволяет сохранить размеры входного изображения, тогда как "valid padding" не добавляет нулей, что приводит к уменьшению размерности.

Настройка скорости обучения и параметров оптимизатора

Гиперпараметры оптимизатора, такие как скорость обучения, коэффициенты затухания (например, β_1 и β_2 в Adam) и размер батча, играют важную роль в процессе обучения CNN. Небольшая скорость обучения позволяет избежать перескоков через минимум функции потерь, в то время как слишком маленькая может привести к медленной сходимости. Подбор этих параметров зачастую осуществляется с использованием методов Grid Search или адаптивных алгоритмов оптимизации.

Пример реализации комплексной CNN с использованием Keras

```
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv2D, BatchNormalization, Activation, MaxPooling2D,
GlobalAveragePooling2D, Dense, Dropout, Concatenate

input_tensor = Input(shape=(64, 64, 3))

# Первый сверточный блок
x = Conv2D(32, (3, 3), padding='same')(input_tensor)
x = BatchNormalization()(x)
x = Activation('relu')(x)
x = MaxPooling2D(pool_size=(2, 2))(x)

# Второй сверточный блок
x = Conv2D(64, (3, 3), padding='same')(x)
x = BatchNormalization()(x)
x = Activation('relu')(x)
x = MaxPooling2D(pool_size=(2, 2))(x)

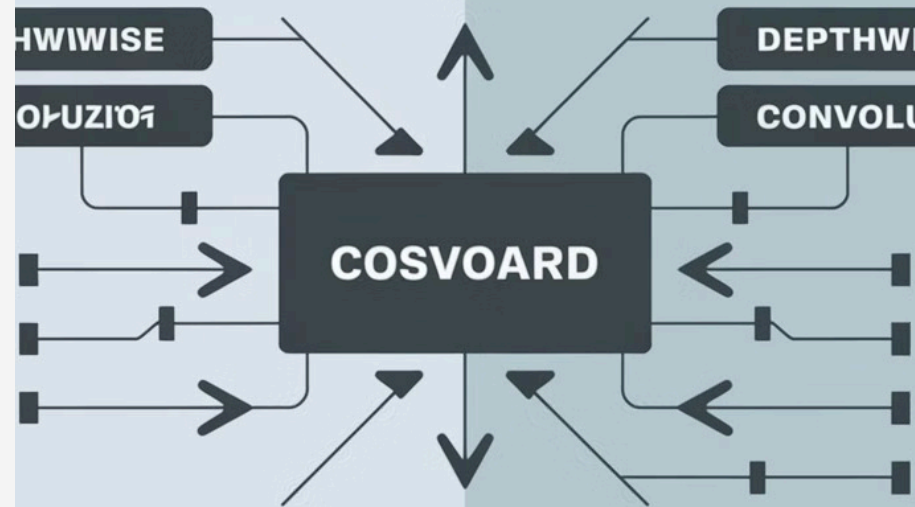
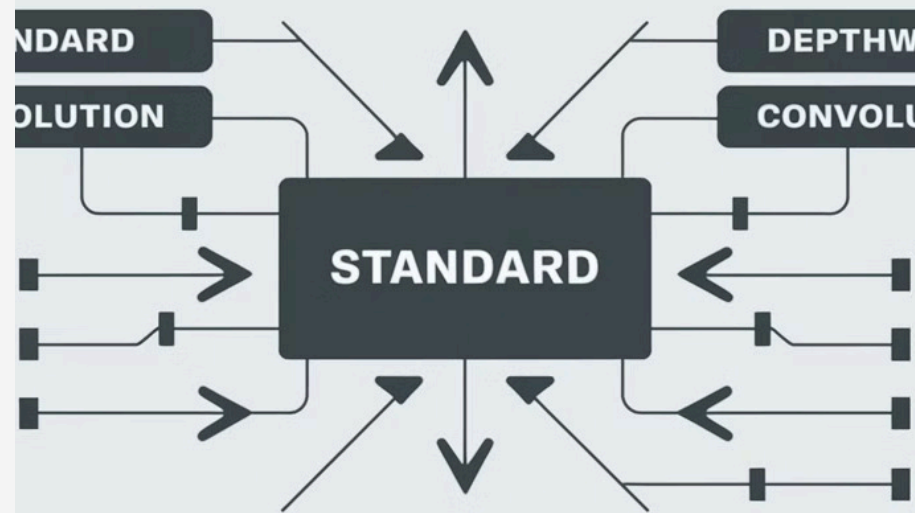
# Третий сверточный блок с использованием групповых сверток
branch1 = Conv2D(128, (3, 3), padding='same', activation='relu')(x)
branch2 = Conv2D(128, (5, 5), padding='same', activation='relu')(x)
x = Concatenate()([branch1, branch2])
x = BatchNormalization()(x)
x = MaxPooling2D(pool_size=(2, 2))(x)

# Глобальный average pooling
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
output_tensor = Dense(10, activation='softmax')(x)

model = Model(inputs=input_tensor, outputs=output_tensor)
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

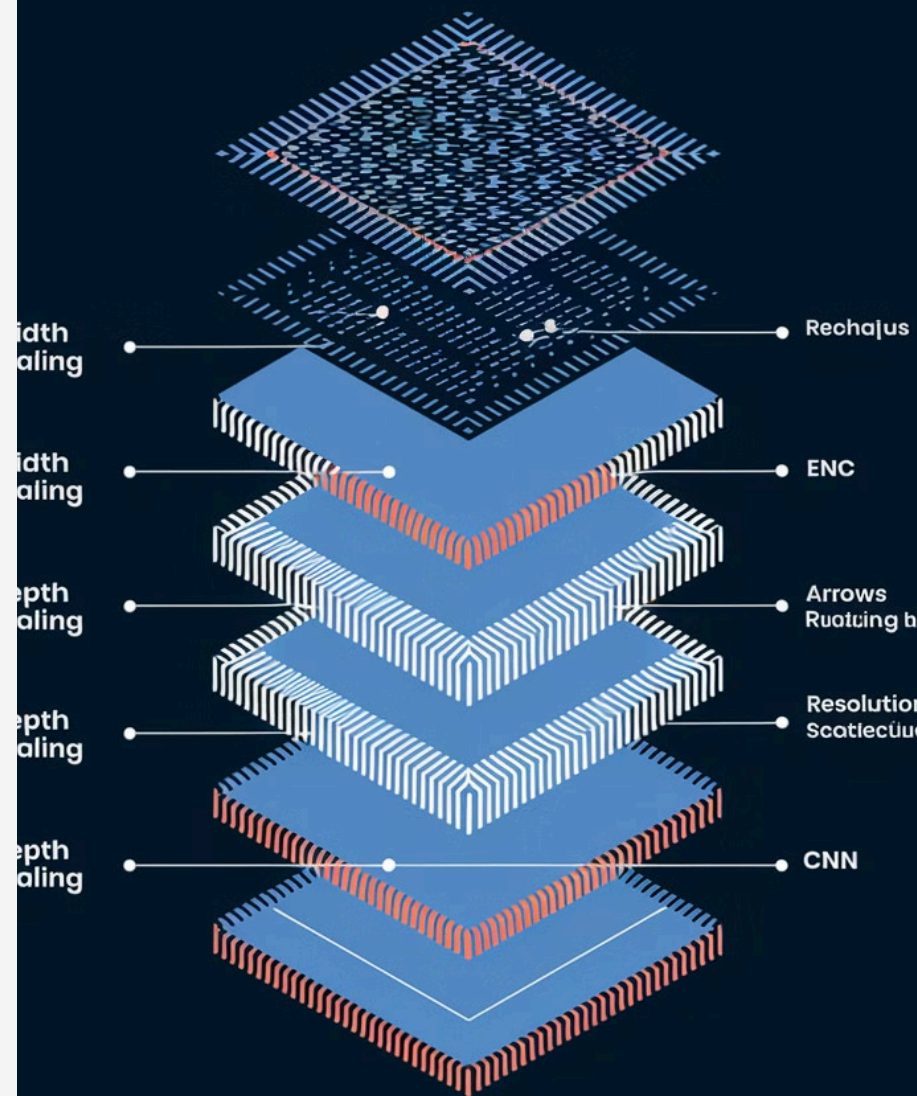
Снижение вычислительной нагрузки

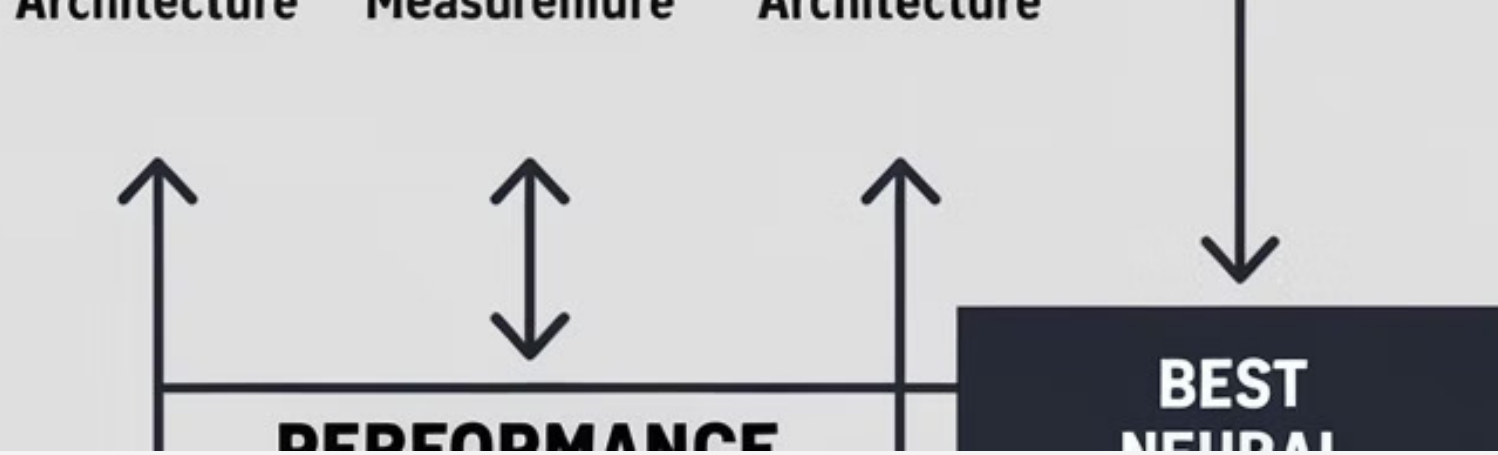
- 1 Использование разделяемых сверточных слоев, где фильтры применяются отдельно к каждому каналу с последующим объединением.
- 2 Применение глубинно-сепарабельных сверток (depthwise separable convolutions), которые разбивают стандартную свертку на два этапа: сначала происходит поэлементная свертка по каждому каналу, затем объединение результатов с помощью точечной (1×1) свертки.



Масштабирование архитектур CNN

При построении очень глубоких сетей важную роль играют методы, позволяющие эффективно масштабировать архитектуру без значительного увеличения вычислительной сложности. Остаточные связи (residual connections) и модульные архитектуры (например, Inception) позволяют увеличивать глубину сети, сохраняя при этом стабильность обучения.



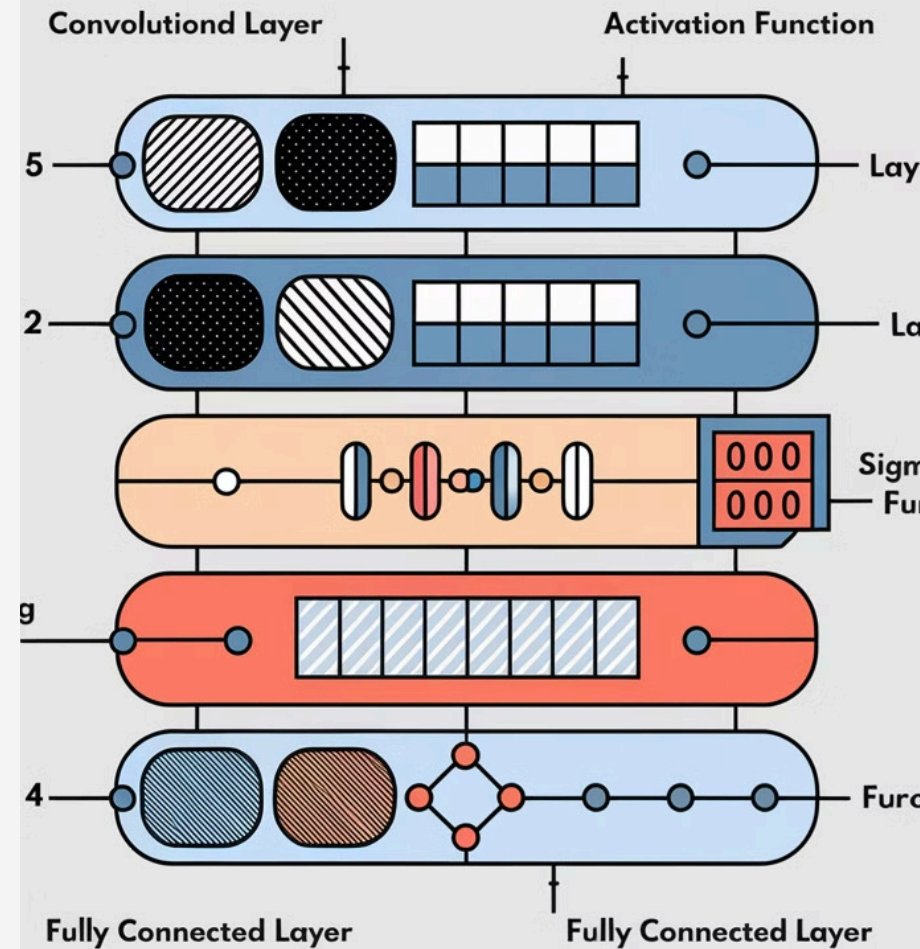


Динамическая настройка архитектуры

Некоторые современные подходы предусматривают адаптивное изменение архитектуры сети в зависимости от сложности входных данных. Например, нейроэволюционные алгоритмы или методы автоматизированного поиска архитектур (Neural Architecture Search, NAS) позволяют оптимизировать структуру сети, выбирая оптимальное количество слоев, фильтров и параметров для конкретной задачи.

Ключевые принципы работы CNN

Сверточные нейронные сети основаны на локальной обработке изображений посредством сверток, что позволяет эффективно выявлять пространственные зависимости и иерархические признаки. Основные компоненты — сверточные слои, функции активации, пулинг и полносвязные слои — в совокупности образуют мощный инструмент для решения задач компьютерного зрения.





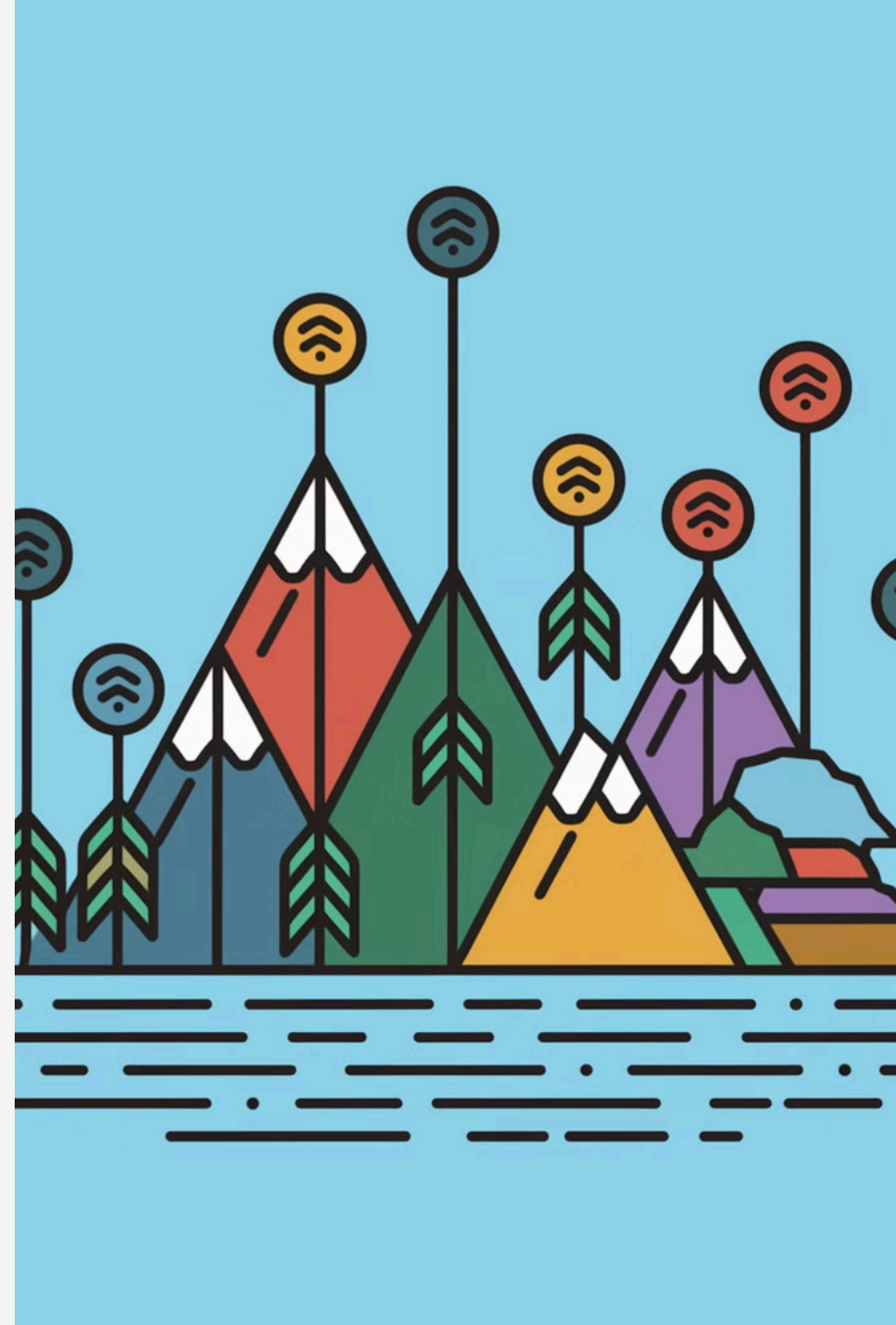
Математическая строгость CNN

Математическое описание операций свертки, нормализации и оптимизации в CNN обеспечивает глубокое понимание процессов обучения и позволяет проводить детальный анализ работы сети. Вычисление градиентов, применение метода обратного распространения ошибки и использование адаптивных оптимизаторов представляют собой фундамент для разработки новых архитектур.

Практическая реализация CNN

Реализация CNN с использованием современных фреймворков (TensorFlow, PyTorch) демонстрирует высокую эффективность глубоких нейронных сетей в задачах классификации, сегментации и детектирования объектов.

Интеграция методов регуляризации, аугментации данных и оптимизации вычислительных процессов позволяет создавать модели, способные работать в режиме реального времени на больших наборах данных.



Активационные карты и визуализация фильтров

Для глубокого понимания работы CNN важно изучать, какие признаки извлекаются на различных слоях сети. Активационные карты (activation maps) представляют собой выходы сверточных слоев до применения операции пулинга или перед передачей в полносвязанные слои.

Визуализация таких карт позволяет наблюдать, какие участки изображения активируются фильтрами, что помогает интерпретировать, какие особенности (края, текстуры, формы) важны для модели.



Методы интерпретации: saliency maps, Grad-CAM, occlusion sensitivity

1

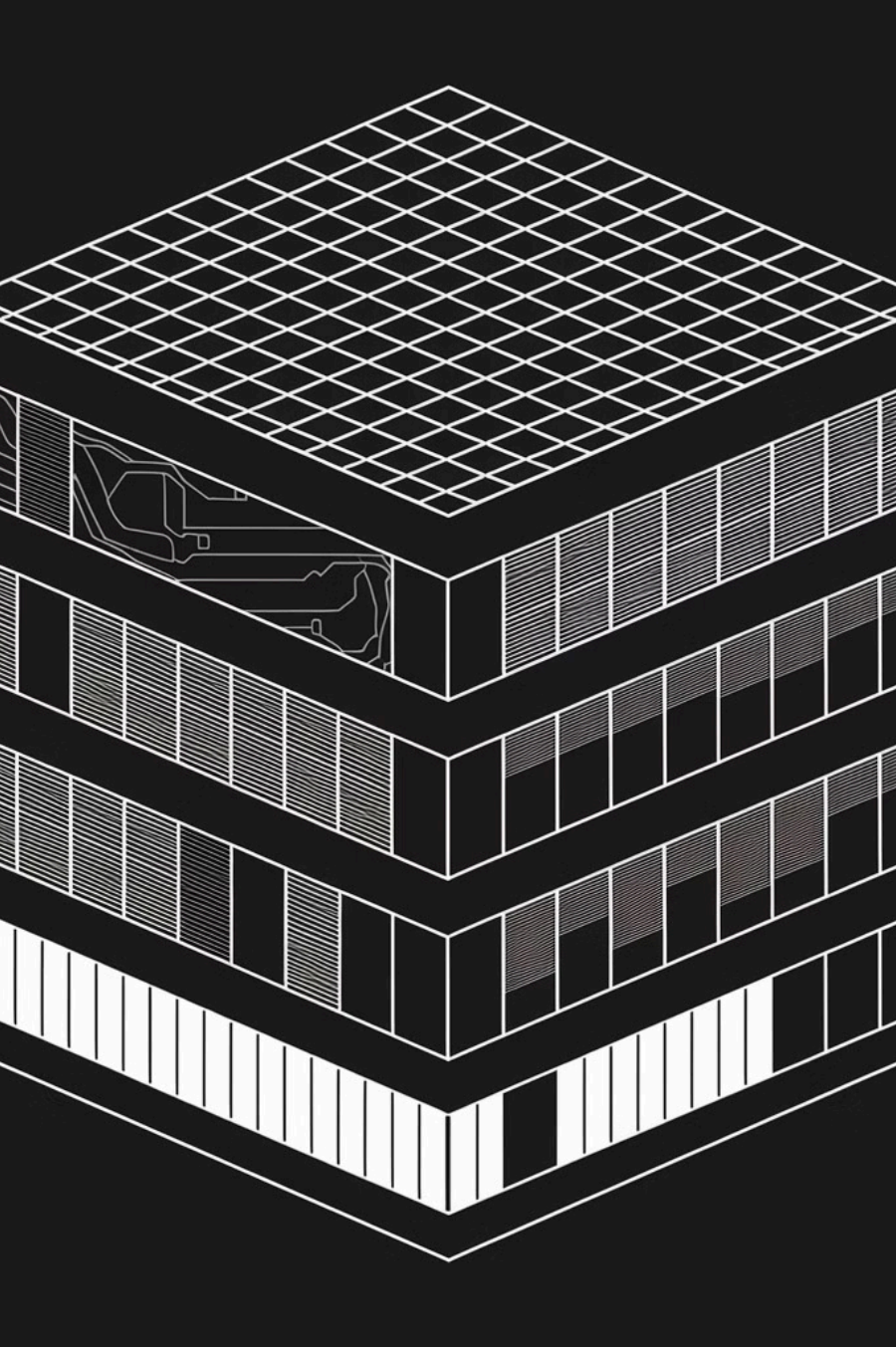
Saliency maps: Метод вычисляет градиенты функции потерь по входному изображению.

2

Grad-CAM: Использует градиенты, полученные от выходного слоя, для вычисления важности признаков карт последнего сверточного слоя.

3

Occlusion sensitivity: Заключается в систематическом затемнении различных областей входного изображения с последующим анализом изменения выходного предсказания.

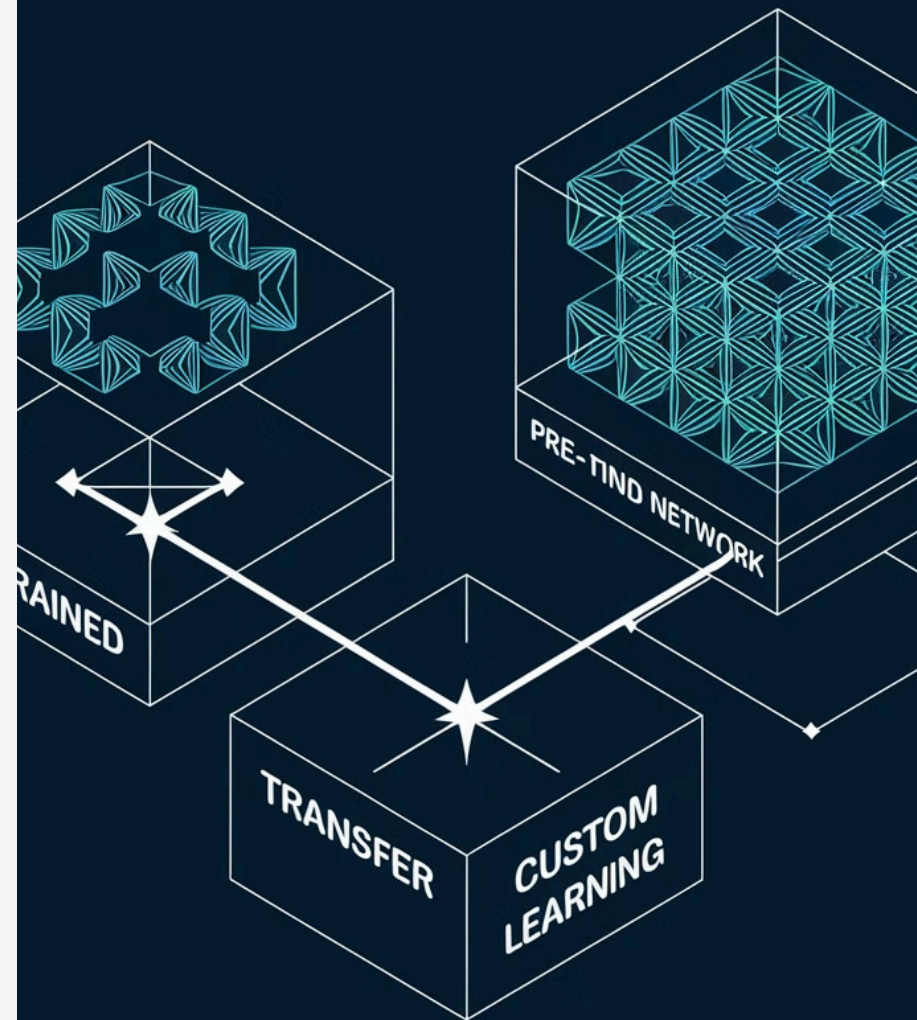


Визуализация фильтров и пространственных признаков

Анализ весов фильтров позволяет оценить, какие паттерны они способны обнаруживать. Для первого слоя сети часто наблюдаются фильтры, выделяющие края, линии и простейшие текстурные паттерны. Визуализация фильтра К может быть выполнена посредством интерпретации его значений как изображения.

Понятие трансферного обучения

Трансферное обучение (transfer learning) предполагает использование модели, предварительно обученной на одном большом наборе данных, для решения новой задачи с ограниченным количеством размеченных примеров. Основная идея заключается в том, что низкоуровневые признаки, извлекаемые CNN, являются универсальными и могут быть полезны для различных задач компьютерного зрения.



Методы трансферного обучения: заморозка слоев и fine-tuning

Заморозка слоев

Начальные слои, обученные на большом датасете, остаются неизменными, а только верхние слои адаптируются под новую задачу.

Fine-tuning

Подразумевает дообучение некоторых из этих слоев с использованием более низкой скорости обучения.



Пример использования предобученной модели в Keras

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D

# Загрузка модели VGG16 с предобученными весами
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Заморозка базовых слоев
for layer in base_model.layers:
    layer.trainable = False

# Добавление новых слоев для целевой задачи
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
predictions = Dense(10, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=predictions)

model.compile(optimizer=tf.keras.optimizers.Adam(lr=0.001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

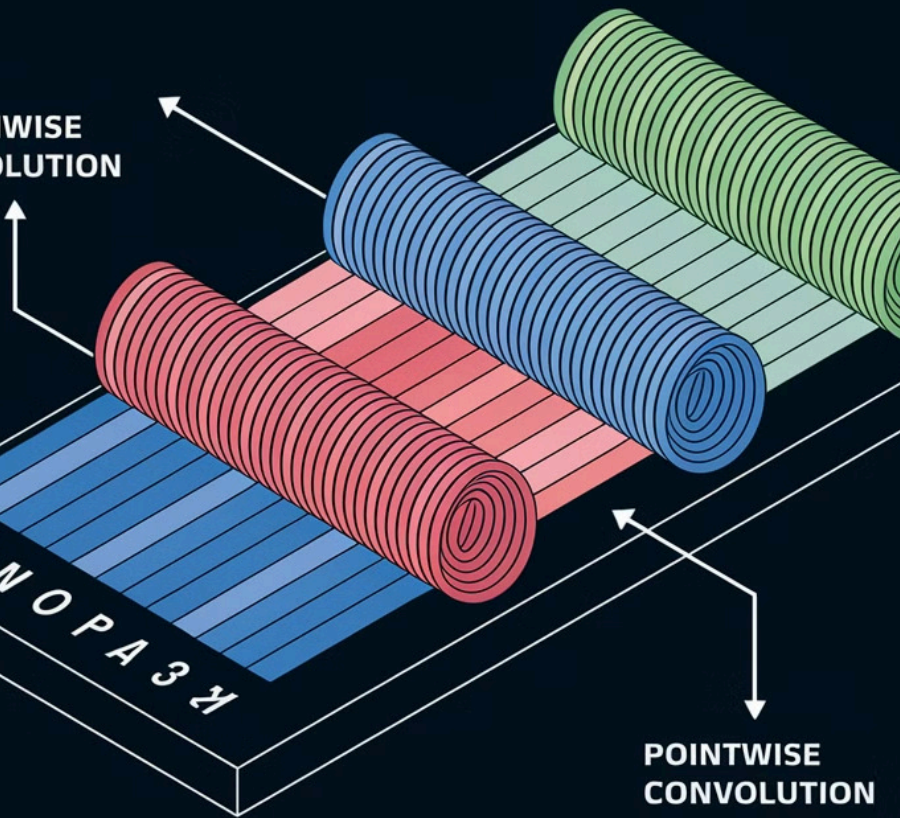
Преимущества и ограничения трансферного обучения

Преимущества

Возможность добиться высоких результатов даже при ограниченном количестве данных, а также сокращение времени обучения.

Ограничения

Предварительно обученная модель может не идеально соответствовать специфике целевой задачи, что требует адаптации архитектуры или дополнительных методов fine-tuning.



Глубинно-сепарабельные свертки

Глубинно-сепарабельные свертки (depthwise separable convolutions) представляют собой метод разбиения стандартной свертки на два этапа: сначала выполняется свертка по каждому входному каналу (depthwise convolution), а затем результаты объединяются с помощью точечной свертки (pointwise convolution, 1×1 свертка).

Математически это можно записать как:

$$S(i,j,c') = \sum_{c=1 \text{ to } C} [\sum_{(u,v)} I(i+u,j+v,c) \cdot K_{dw}(u,v,c)] \cdot K_{pw}(c,c')$$

где K_{dw} – фильтр для depthwise свертки, а K_{pw} – фильтр для точечной свертки.

Прореживание (pruning) и квантование (quantization)

Прореживание (pruning)

Метод, при котором незначимые веса модели обнуляются, что приводит к разреженной структуре сети.

Квантование (quantization)

Приведение весов и активаций к более низкой точности (например, 8-битное представление вместо 32-битного), что снижает требования к памяти и ускоряет вычисления.



Knowledge Distillation

Knowledge distillation представляет собой метод переноса знаний от крупной, высокоточной модели (teacher) к более компактной модели (student). Идея заключается в том, чтобы минимизировать разницу между предсказаниями teacher и student, используя, например, Kullback–Leibler divergence в качестве функции потерь:

$$L_{KD} = KL(T(y) \parallel S(y))$$

где $T(y)$ и $S(y)$ – распределения вероятностей, предсказанные teacher и student соответственно.

Параллельные вычисления и распределённое обучение

1

Data Parallelism: Разделение обучающей выборки на мини-батчи, обрабатываемые параллельно на нескольких GPU, с последующей синхронизацией градиентов.

2

Model Parallelism: Разделение архитектуры сети на части, каждая из которых обучается на отдельном устройстве.

3

Синхронное и асинхронное обновление параметров: Выбор между синхронной и асинхронной схемами обновления влияет на скорость и стабильность сходимости модели.



